NvM Job Queue / State Machine

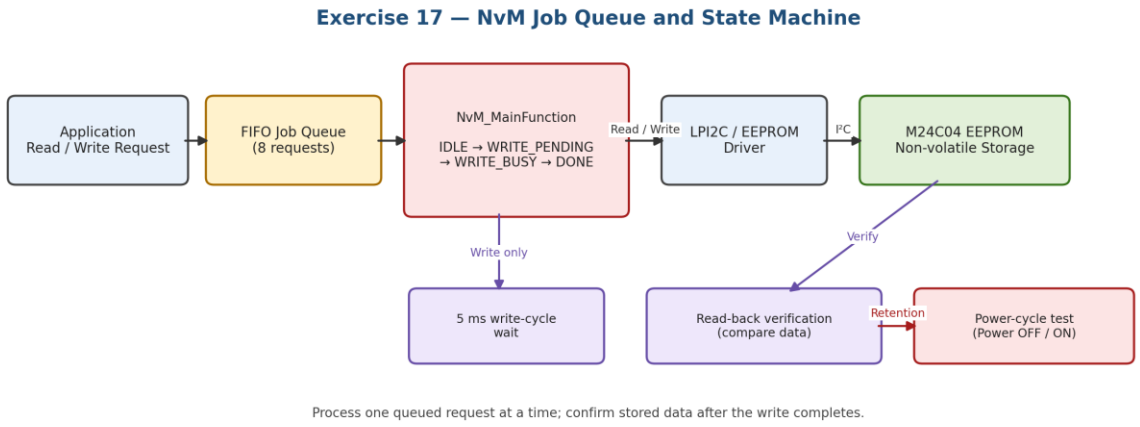
PROJECT DOCUMENTATION

Tool:

- S32 Design Studio
- PuTTY (UART monitoring)

Interfaces:

- I²C EEPROM
- UART (PuTTY)



1. Overview

This project implements a simplified AUTOSAR-style Non-Volatile Memory (NvM) Manager using an NXP S32K144 microcontroller and an external M24C04 EEPROM. Multiple write requests are stored in a First-In, First-Out (FIFO) queue and processed sequentially through a state machine. The program verifies stored data through read-back comparison, and a separate power-cycle test checks whether the data remains available after power is removed and restored.

The main state sequence is IDLE → WRITE_PENDING → WRITE_BUSY → DONE. NvM_MainFunction() advances the state machine, and UART output is used to observe job processing and verification results.

2. Requirements and scope

Rn	Requirement
R1	Accept EEPROM write requests from multiple application blocks.
R2	Store requests in a FIFO queue and process them in acceptance order.
R3	Implement the IDLE → WRITE_PENDING → WRITE_BUSY → DONE state sequence.
R4	Use I²C/LPI2C to communicate with the external M24C04 EEPROM.
R5	Verify written data by reading it back and comparing it with the expected data.
R6	Provide a separate power-cycle retention test.
R7	Report state transitions and verification results through UART.

3. How it works (simple view)

1. Power on and initialize the clock, pins, UART, and LPI2C peripherals.
2. Submit EEPROM write requests from the application.
3. Store each accepted request in the FIFO job queue.
4. NvM_MainFunction() retrieves the next job and starts the EEPROM write operation.
5. Wait for the EEPROM write cycle to complete (approximately 5 ms).
6. Read the stored data back and compare it with the expected value.
7. Remove the completed job from the queue and process the next request.
8. Report the result through UART.

A separate power-cycle test removes and restores power, then reads the stored data again without rewriting it.

4. EEPROM memory and request data

Each queued request identifies the EEPROM block or address, the data buffer, and the data length. The queue stores up to eight requests, according to the project configuration.

Request field	Description
Block ID / address	Identifies the EEPROM location to access.
Data buffer	Contains the bytes to write or receives the bytes read.
Data length	Number of bytes involved in the request.

5. Test data

The application submits three write requests for EEPROM blocks 1, 2, and 3. Each block contains four bytes of test data.

EEPROM Block	Expected data	Initial read-back result
Block 1	10, 20, 30, 20	PASS
Block 2	50, 60, 70, 80	PASS
Block 3	90, 100, 110, 120	PASS

6. Software structure

File	Function / purpose
main.c	Initializes peripherals, submits requests, monitors states, and verifies stored data.
nvm.c	Implements the NvM state machine and coordinates write operations.
queue.c	Implements FIFO enqueue and dequeue operations.
eeprom.c	Implements I²C-based EEPROM read and write operations.
nvm.h	Defines NvM states, data structures, constants, and function declarations.
eeprom.h	Declares EEPROM functions and status types.

7. NvM state machine

The state machine processes queued write requests using the following sequence:

IDLE → WRITE_PENDING → WRITE_BUSY → DONE → IDLE

State	Purpose
IDLE	Checks whether the queue contains a request.
WRITE_PENDING	Starts the EEPROM write operation.
WRITE_BUSY	Waits for the EEPROM internal write cycle (approximately 5 ms).
DONE	Records the result, removes the completed job, and returns to IDLE.

NvM_MainFunction() is called repeatedly to advance the state machine. The write-cycle wait should not block the main loop. Read requests can use the same queueing approach, but do not normally need the EEPROM's internal write-cycle delay.

8. EEPROM interface

Item	Description
Interface	I²C using the S32K144 LPI2C peripheral.
EEPROM device	M24C04 external EEPROM.
Write-cycle delay	Approximately 5 ms after a write.
Read verification	Read back stored bytes and compare with expected data.
Power removal	Only after the write cycle has completed.

The current write-cycle handling uses a fixed delay rather than I²C ACK polling. EEPROM communication errors should be reported as failed jobs rather than treated as successful completion.

9. Testing and verification

9.1 Initial read-back verification

1. Initialize the microcontroller peripherals.
2. Submit the three EEPROM write requests.
3. Allow the state machine to process each request sequentially.
4. Wait for the write operations to complete.
5. Read back blocks 1, 2, and 3.
6. Compare the retrieved bytes with the expected data.
7. Confirm the PASS or FAIL result through UART.

9.2 Power-cycle verification

1. Complete the initial write and read-back test.
2. Ensure the EEPROM write cycle has completed.
3. Remove power from the EEPROM and the board as required by the hardware setup.
4. Restore power and restart the microcontroller.
5. Initialize LPI2C and UART.
6. Read the EEPROM blocks without rewriting the test data.
7. Compare the retrieved data with expected values stored independently in firmware.
8. Report PASS if the values match; otherwise report FAIL.

Important: If startup code automatically rewrites the test pattern, disable that behavior during the power-cycle test. Power-cycle retention should only be reported as successful after the test is performed and the UART result is recorded.

10. Algorithm decision table

Condition	Expected result
Queue is empty	Remain in IDLE and wait for a request.
A write request is available	Start the write and enter the write-cycle wait state.
Write cycle completes	Mark the job complete and remove it from the queue.
Read-back data matches	Report verification PASS.
Read-back data differs	Report verification FAIL.
I²C transaction fails	Report a failed job and handle the error.

11. Build and execution

1. Open the project in NXP S32 Design Studio.
2. Check clock configuration, pin multiplexing, LPUART, and LPI2C settings.
3. Confirm EEPROM wiring and I²C address configuration.
4. Build the project and resolve compilation errors.
5. Flash the firmware to the S32K144 evaluation board.
6. Open the configured UART COM port using PuTTY.
7. Observe queue processing, state transitions, write results, and read-back verification.
8. Perform the power-cycle test and run read-only verification after restart.

12. Expected UART output and results

UART monitoring should report queue activity, state transitions, EEPROM operation status, and data verification.

The supplied project notes report the following initial read-back results:

Block	Expected data	Result
1	10, 20, 30, 20	PASS
2	50, 60, 70, 80	PASS
3	90, 100, 110, 120	PASS

The power-cycle test is separate. Its result must be recorded after power is removed and restored; the initial read-back results do not establish power-cycle retention.

13. Conclusion

This project demonstrates a simplified AUTOSAR-style NvM Manager using a FIFO job queue, a state machine, and an external M24C04 EEPROM over I²C. It covers sequential request handling, EEPROM write cycle management, UART monitoring, read-back verification, and a separate procedure for verifying non-volatile data retention after power removal and restoration.